Vertaling van MPLang naar MPNN

Nette Beckers
Universiteit Hasselt
Hasselt, Belgium
nette.beckers@student.uhasselt.be

***Abstract*—Graph Neural Networks (GNN's) zijn een klasse van neurale netwerken die specifiek zijn ontworpen voor het werken met grafen. We verdiepen ons in Message Passing Neural Networks (MPNN's), een subklasse van GNN's die feature maps transformeren via sequentiële lagen. Een nieuwe taal, MPLang, geïntroduceerd door Prof. Dr. Jan Van den Bussche en Juno Steegmans, vereenvoudigt de operaties binnen MPNN's. We bespreken de theoretische vertaling van MPLang expressies naar MPNN sequenties en presenteren een algoritme voor deze vertaling. Dit werk heeft als doel een gebruiksvriendelijke taal te creëren voor het structureren van MPNN's en deze effectief te implementeren in een populair machine learning framework.**

I. Introductie

Graph Neural Networks (GNN's) zijn vandaag zeer populair binnen de ontwikkeling van machine learning. Het zijn neurale netwerken die nieuwe informatie over grafen kunnen berekenen. In deze paper gaan we verder in op een subklasse van GNN's, namelijk de message passing neural networks (MPNN's). [1]

Deze MPNN's maken gebruik van de communicatie tussen buren in een graaf, die hun informatie met elkaar delen. Hierdoor kunnen er belangrijke nieuwe voorspellingen worden gedaan voor een node, edge of voor een graaf zelf.

Een MPNN is gestructureerd in lagen en zijn zeer complex te construeren. Hiervoor hebben Prof. Dr. Jan Van den Bussche en Juno Steegmans, een nieuwe taal ontwikkeld namelijk MPLang. MPLang maakt het mogelijk eenvoudige expressies te maken, die kunnen worden vertaald naar MPNN's.

In deze paper zullen we de vertaling van MPLang naar MPNN implementeren in Python. Python is gekozen, omdat het veel wordt gebruikt binnen de machine learning. Ook zijn er daarom handige libraries binnen Python, om deze implementatie mogelijk te maken.

De paper is gestructureerd als volgt: Er worden eerst basisconcepten uitgelegd, zoals: GNN in sectie II, grafen en feature maps in sectie III, Global Feature Map Transformers in sectie IV en MPLang in sectie VI. Daarna gaan we over tot de probleemstelling in sectie VII en kan het algoritme VIII worden beschreven. Als laatste komen we bij de implementatie in sectie X van het algoritme en de keuzes in sectie IX die er gemaakt zijn.

II. Graph Neural Network (GNN)

Een GNN is een klasse van neurale netwerken, dat specifiek werkt op gegevens die gestructureerd zijn in grafen. De noden en edges hebben informatie. De GNN gaat nieuwe informatie voor noden en edges genereren, door de structuur van de graaf te analyseren. [1], [5]

Een voorbeeld ter verduidelijking is een sociaal netwerk. Elke node is een persoon en heeft enkele kenmerken zoals leeftijd, geslacht, woonplaats, etc. De edges zijn de relaties tussen deze personen, bijvoorbeeld kennen ze elkaar of niet. Nieuwe informatie die bijvoorbeeld zou kunnen gegenereerd worden is een voorspelling nieuwe relaties die gemaakt zullen worden. Dit wordt in sociale media vaak toegepast onder personen die u misschien kent. Dit is een vorm van edge prediction, we gaan nieuwe edges voorspellen.

We kunnen ook node classificeren, waarbij we iets over een node gaan voorspellen. Zoals bijvoorbeeld sociale media maken hier gebruik van voor reclame. Aan de hand van alle informatie kunnen ze gaan voorspellen welke persoon welk product zou aankopen.

Grafen of delen van grafen gaan we ook kunnen classificeren. Bij sociale netwerken kan dit gaan over het voorspellen van gemeenschappen, zoals iedereen van een bepaalde regio of dezelfde interesses.
Dit is maar een klein voorbeeld, in de wetenschap is het ook zeer populair. Het onderzoek naar GNN's is in de laatste jaren gestegen, waardoor ze steeds verder ontwikkeld worden. In de chemie, worden GNN's gebruikt om voorspellingen te maken rond moleculen. De mogelijkheden met GNN's zijn dus groot en veel belovend voor de toekomst.

III. Grafen en Feature maps

Een graaf $G = (V, E)$ is een geordend paar van 2 verzamelingen, de verzameling van de vertices (nodes) V en van de edges E. Features zijn informatie over een node, die in ons geval als numerieke waarden worden bijgehouden. We hebben een feature map van een graaf, die alle featurevectoren mapt naar de nodes. De featurevector van een node is een eindige vector met de kenmerken.

We gaan ervan uit dat de grafen ongericht zijn en dus zonder loops, dit wil zeggen dat we spreken over symmetrische antireflexieve grafen.

Als er voor een natuurlijk getal d (dit is het aantal features) een d-dimensionale feature map op G is, kunnen we de functie die de nodes mapt naar de feature vectoren beschrijven als volgt: $\chi : V(G) \to \mathbb{R}^d$. De set van alle mogelijke d-dimensionale feature maps op G noteren we als $Feat(G, d)$.

IV. Global Feature Map Transformer (GFMT)

Een Global Feature Map Transformer (GFMT) kan operaties uitvoeren op grafen, die de features gaat transformeren. Global functies zijn geïntroduceerd door Gurevich, met als doel uniform definiëren van functies in structuren. De GFMT, geïntroduceerd door prof. dr. Jan Van de Bussche en Juno Steegmans, is geïnspireerd op de Global functies van Gurevich met als doel uniforme functies te definiëren op feature maps. Deze sectie is volledig voortgebracht uit de onderzoekspaper, [2].

Neem r een natuurlijk getal, een GFMT T transformeert een d-dimensionale feature map naar een r-dimensionale feature map. De GFMT T heeft dan als type $d \to r$, d en r worden ook wel de input en output ariteit genoemd. Bijvoorbeeld een GFMT T_1 van type $2 \to 1$ dat het gemiddelde neemt van de eerste twee feature waarden. Met $\chi(v)[i]$ kunnen we het i-de component uit de featurevector van node v halen. Dus de GFMT is $T_1(G)(\chi)(v) = (\chi(v)[1] + \chi(v)[2])/2$. De input aritieit is hierbij 2, want we krijgen per node twee features, merk op dit wil ook zegggen dat hierbij $d = 2$. De output ariteit is $r = 1$, we krijgen per node één feature terug. De r-dimensionale output feature map op G is dan $T(G)(\chi)$.

In deze paper zal er vaak gewerkt worden met GFMT's van output ariteit 1. Een GFMT met een hogere output ariteit dan 1, kunnen dan worden voorgesteld als tuples van GFMT's met output ariteit 1. Dus een GFMT T van type $d \to r$ kan worden gedefinieerd door een tuple $(T_1, \ldots, T_r)$. Als we deze GFMT dan op een node v uitvoeren krijgen we $T(G)(\chi)(v) : (T_1(G)(\chi)(v), \ldots, T_r(G)(\chi)(v))$.

Op deze GFMT's kunnen we operaties gaan uitvoeren, zoals:

- **Concatenatie:** Met 2 GFMT's T_1 en T_2, met als type $d \to r_1$ en $d \to r_2$. Dan wordt de concatenatie $L_1|L_2$ van type $d \to r_1 + r_2$, de input is hetzelfde als voor één van de lagen. De output daarin tegen is de concatenatie van de twee outputs van de lagen. We kunnen de bovenstaande tuple dus herschrijven naar $T_1|\ldots|T_r$, zo krijgen we een GFMT van hogere output ariteit.
- **Parallelle compositie:** $T_1||T_2$ van 2 GFMT's T_1 en T_2 van type $d_1 \to r_1$ en $d_2 \to r_2$ vormt een GFMT T van type $(d_1 + d_2) \to (r_1 + r_2)$. De input van beide GFMT's wordt gestapeld voor de input van de parallelle compositie. De GFMT wordt gedefinieerd door $(T_1||T_2)(G)(\chi)(v) = T_1(G)(\chi_1)(v)|T_2(G)(\chi_2)(v)$.
- **Sequentiële compositie:** $T_1;T_2$ van 2 GFMT's T_1 en T_2 van type $d_1 \to d_2$ en $d_2 \to d_3$ vormt een GFMT van type $d_1 \to d_3$. De GFMT's worden achter elkaar gezet en de output van de GFMT T_1 dient als de input voor GFMT T_2. De GFMT wordt gedefinieerd door $(T_1;T_2)(G)(\chi)(v) = T_2(G)(T_1(G)(\chi))(v)$.
- **Som:** $T_1 + T_2$ van 2 GFMT's T_1 en T_2 van type $d \to r$ vormt een GFMT T van type $d \to r$. De outputs van beide GFMT's T_1 en T_2 worden opgeteld en vormen de output van de GFMT T_1+T_2. Die gedefinieerd wordt door $(T_1 + T_2)(G)(\chi)(v) = T_1(G)(\chi_1)(v) + T_2(G)(\chi_2)(v)$.
- **Verschil:** $T_1 - T_2$ van 2 GFMT's T_1 en T_2 van type $d \to r$ vormt een GFMT T van type $d \to r$. Het verschil van de outputs van GFMT's T_1 en T_2 vormen de output van de GFMT $T_1 - T_2$. Die gedefinieerd wordt door $(T_1 - T_2)(G)(\chi)(v) = T_1(G)(\chi_1)(v) - T_2(G)(\chi_2)(v)$.

Ter verduidelijking zijn hier nog enkele voorbeelden van de operaties met GFMT's. We gebruiken de inputgraaf $G = (V, E)$ met de adjacencymatrix

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

en de feature map

$$\left[\begin{bmatrix} 12 \\ 2 \\ 7 \\ 3 \end{bmatrix} \begin{bmatrix} 10 \\ 1 \\ 15 \\ 2 \end{bmatrix} \begin{bmatrix} 3 \\ 8 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 25 \\ 4 \\ 1 \\ 5 \end{bmatrix} \right]$$

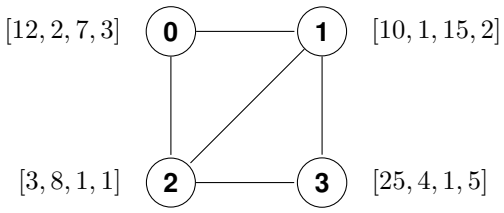


Fig. 1. Voorbeeld graaf G

Stel GFMT $L_1(G)(\chi)(v) = (\chi(v)[1] + \chi(v)[2])$, GFMT $L_2(G)(\chi)(v) = max(\chi(v)[1], \chi(v)[2])$, beide van type $4 \to 1$. Het type zou ook $2 \to 1$ kunnen zijn, als er maar twee features zijn bij de input feature map. Stel ook nog GFMT's die van type $1 \to 1$ kunnen zijn, namelijk GFMT $L_3(G)(\chi)(v) = 2 \cdot (\chi(v)[1])$ en GFMT $L_4(G)(\chi)(v) = max(\chi(v)[1], 12)$, dan:

- **Concatenatie $L_1|L_2$:** Geeft voor één node de featurevector terug met beide outputs. De output van de GFMT $L_1|L_2$ is dan $\left[\begin{bmatrix} 14 \\ 12 \end{bmatrix} \begin{bmatrix} 11 \\ 10 \end{bmatrix} \begin{bmatrix} 11 \\ 8 \end{bmatrix} \begin{bmatrix} 29 \\ 25 \end{bmatrix} \right]$.
- **Parallelle compositie $L_1||L_3$:** Geeft voor één node de featurevector terug met beide outputs. Dit lijkt hetzelfde als bij de concatenatie, maar merk op dat de input voor

de GFMT $L_1||L_3$ de twee inputs voor beide GFMT's L_1 en L_3 stapelt. Dit wordt in de sectie van het algoritme nog duidelijk. De output van de GFMT $L_1||L_3$ is dan $\left[\begin{bmatrix}14\\24\end{bmatrix}\begin{bmatrix}11\\20\end{bmatrix}\begin{bmatrix}11\\6\end{bmatrix}\begin{bmatrix}29\\50\end{bmatrix}\right]$.

- **Sequentiële compositie** $L_2; L_4$ Geeft de output van GFMT L_2 door als input aan GFMT L_4. Voor één node krijgen we dan eerst de som van de eerste twee features, daarna wordt het maximum genomen van dit resultaat en 12. De output voor L_2 en de input voor L_4 is dan $\begin{bmatrix}14 & 11 & 11 & 29\end{bmatrix}$. De output van de GFMT $L_2; L_4$ wordt dan $\begin{bmatrix}14 & 12 & 12 & 29\end{bmatrix}$.
- **Som** $L_1 + L_2$ Geeft de som van de output van beide GFMT's L_1en L_2. De output voor GFMT $L_1 + L_2$ wordt dan $\begin{bmatrix}26 & 21 & 19 & 54\end{bmatrix}$.
- **Verschil** $L_1 - L_2$ Geeft het verschil van de output van beide GFMT's L_1en L_2. De output voor GFMT $L_1 - L_2$ wordt dan $\begin{bmatrix}2 & 1 & 3 & 4\end{bmatrix}$.

V. Message Passing Neural Network (MPNN)

Een Message Passing Neural Network (MPNN) komt uit de klasse van GNN's, en gebruikt de functie van het verzenden van informatie tussen buren in de graaf. Een MPNN neemt een graaf als input, met bij elke node een feature vector. De MPNN transformeert de set van feature vectoren op de input graaf naar een nieuwe set feature vectoren op de input graaf.

Deze transformatie gebeurt door middel van meerdere MPNN-lagen. Een enkele laag in een MPNN is gedefinieerd door een 4-tuple $L = (W_1, W_2, b, \sigma)$, waarbij:

- $\sigma : \mathbb{R} \to \mathbb{R}$ een continue functie is, die ook wel de activationfunctie wordt genoemd. In dit geval wordt de laag L dan ook een σ-laag genoemd. De activationfunctie wordt toegepast op elk component van de resulterende vector. Voor deze implementatie wordt ReLu (Rectified Lineair Unit) functie gebruikt, behalve in de laatste laag.
- W_1, W_2 zijn reële matrices van dimensie $r \times d, r \times d$. Deze matrices worden gebruikt om de feature vectoren te transformeren, W_1 voor de huidige node en W_2 voor de buren van de huidige node.
- b is een bias vector met dimensie $r \times 1$. Elk component van deze vector is gelijk en wordt toegevoegd aan de resulterende vector na de transformaties en sommatie van W_1 en W_2.

Een MPNN-laag definieert telkens een Global Feature Map Transformer (GFMT). De transformatie die door een MPNN-laag wordt uitgevoerd, kan worden beschreven als:

$$L(G)(\chi)(v) := \sigma(W_1\chi(v) + W_2 \sum_{u \in N(G)(v)} \chi(u) + b)$$

Waarbij:

- G de inputgraaf is.
- χ de functie die aan elke node v in de graaf G een feature vector toekent.
- $N(G)(v)$ is de verzameling van de buren van de node v.
- σ is de activationfunctie die op elk element van de vector wordt toegepast.
- W_1 en W_2 zijn de transformatie matrices voor de feature vectoren van de huidige node en de som van de buren.
- b is de bias vector.

De werking van een laag gaat als volgt: de feature vector van een node v wordt eerst getransformeerd door de matrix W_1. Daarna wordt W_2 toegepast op de som van de getransformeerde featurevectoren van de buren van v. De resulterende vectoren worden opgeteld en de bias b wordt toegevoegd aan deze som. Ten slotte wordt de activationfunctie σ toegepast op elke element van de resulterende vector.

Een complete MPNN bestaat uit een eindige, niet-lege sequentie van MPNN-lagen, beschreven als $L_1, L_2, ..., L_p$, waarbij p een natuurlijk getal is dat het aantal lagen aangeeft. De input en output ariteit van een laag zijn het aantal features per node bij de input of output. De input ariteit moet dezelfde zijn als de output ariteit van de vorige laag, indien er een vorige laag is. Want de output van elke laag dient als input voor de volgende laag, behalve de eerste laag die de input van de MPNN gebruikt. De MPNN definieert hiermee een GFMT die de sequentiële compositie is van de lagen $L_1; L_2; ...; L_p$.

Voor de activation functie σ zullen we in dit paper de ReLu-functie (rectified lineair unit) gebruiken, met als uitzondering de laatste laag deze heeft de id-functie. Zo een MPNN noemen we dan een ReLu-MPNN, de lagen met een ReLu functie noemen we dan ook een ReLu-laag. De ReLu-functie wordt gebruikt omdat deze zeer populair is binnen de toepassingen van neurale netwerken. ReLu is ook idempotent, dit wil zeggen dat als een waarde eenmaal is getransformeerd door ReLu, verdere toepassingen van ReLu op deze waarde geen verandering meer zullen brengen. Dit is eigenschap die van pas zal komen bij het bewijs voor het concateneren van twee MPNN's.

VI. MPLang

MPLang is een taal geïntroduceerd door prof. dr. Jan Van de Bussche en Juno Steegmans. De taal is ontworpen voor het vereenvoudigen van operaties met neurale netwerken, specifiek de message passing neurale netwerken zoals eerder besproken. Een MPLang expressie creëert een ReLu-MPNN. Deze sectie is volledig voortgebracht uit de onderzoekspaper, [2].

A. Structuur MPLang expressies

In MPLang worden operaties beschreven door expressies. Deze expressies hebben dezelfde input- en outputstructuur als MPNN's. De syntax voor de expressies is gegeven door de grammatica:

$$e ::= 1 \quad | \quad P_i \quad | \quad ae \quad | \quad e + e \quad | \quad f(e) \quad | \quad \Diamond e$$

Waarbij:

- $i \in \mathbb{N}^0$: een natuurlijk getal zonder 0 is.

- $a \in \mathbb{R}$: een reëel getal is.
- $f : \mathbb{R} \to \mathbb{R}$: een continue functie die een reëel getal op een reëel getal afbeeldt.

Een GFMT T van type $d \to 1$ wordt gedefinieerd door een expressie e als volgt:

- **Constante:** als $e = a$, dan $e(G)(\chi)(v) := a$. Dit betekent voor elke node v zal de output de waarde a zijn.
- **Component:** als $e = P_i$, dan $e(G)(\chi)(v) :=$ het i-de component van $\chi(v)$ de featurevector.
- **Scalair product:** als $e = ae_1$, dan $e(G)(\chi)(v) := ae_1(G)(\chi_1)(v)$. De subexpressie e_1 wordt geëvalueerd en daarna geschaald met de factor a.
- **Som:** als $e = e_1 + e_2$, dan $e(G)(\chi)(v) := e_1(G)(\chi_1)(v) + e_2(G)(\chi_2)(v)$. De resultaten van de subexpressies e_1 en e_2 worden opgeteld.
- **Functie:** als $e = f(e_1)$, dan $e(G)(\chi)(v) := f(e_1(G)(\chi)(v)$. De subexpressie e_1 wordt geëvalueerd en daarna wordt de functie f erop toegepast.
- **Diamond:** als $e = \diamond e_1$, dan $e(G)(\chi)(v) := \sum_{u \in N(G)(v)} e_1(G)(\chi)(u)$. De subexpressie e_1 wordt uitgevoerd op alle buren van de node v en de resultaten hiervan worden gesommeerd.

Deze expressies vertrekken steeds vanuit een constante of component expressie. Dit wil zeggen dat de bladeren in de expressieboom steeds een getal of component expressie zullen zijn. Voor al de andere expressies heb je een andere expressie nodig.

B. Voorbeelden expressies

Om een beter idee te krijgen van hoe deze expressies werken, zijn hier enkele voorbeelden. We gebruiken als input, dezelfde graaf en feature map als in de paragraaf van de GFMT's. We verwijzen hiervoor terug naar figuur 1.

- **Constante expressie:** $e_1 = 5$, voor elke node v zal de output $e(G)(\chi)(v)$ gelijk aan 5 zijn. De resulterende feature map is dan $\begin{bmatrix}5 & 5 & 5 & 5\end{bmatrix}$.
- **Component expressie:** $e_2 = P_2$ haalt de tweede component uit de feature vector van een node. Dus dan krijgen we $e_2(G)(\chi)(0) = 2$ voor node 0 bijvoorbeeld, voor e_2 krijgen we dan als output $\begin{bmatrix}2 & 1 & 8 & 4\end{bmatrix}$
- **Scalair product expressie:** $e_3 = 3 \cdot e_2 = 3 \cdot (P_2)$ betekent dat eerst de subexpressie e_2 wordt uitgevoerd, daarna vermenigvuldigt met 3. Voor node 0 krijgen we $e_3(G)(\chi)(0) = 3.(2) = 6$, de output van de expressie e_3 is $\begin{bmatrix}6 & 3 & 24 & 12\end{bmatrix}$.
- **Som expressie:** $e_4 = P_1 + P_2$, telt de eerste en tweede elementen van de featurevectoren van een node v op. We voeren eerste de 2 subexpressies uit, dus dan krijgen we als output van deze subexpressies $\begin{bmatrix}12 & 10 & 3 & 25\end{bmatrix}$ en $\begin{bmatrix}2 & 1 & 8 & 4\end{bmatrix}$. Voor node 0 krijgen we dan $e_4(G)(\chi)(0) = 12 + 2 = 14$, de output van de expressie e_4 is dan $\begin{bmatrix}14 & 11 & 11 & 29\end{bmatrix}$.
- **Functie expressie:** We gebruiken de functie $f(x) = 5 \cdot x$, dus $e_5 = f(e_2) = 5 \cdot e_2 = 5 \cdot P_2$. Op het resultaat van de subexpressie e_2 wordt de functie uitgevoerd. Voor node 0 is dit als volgt $e_5(G)(\chi)(0) = 5 \cdot 2 = 10$. De resulterende feature map van de expressie e_5 is $\begin{bmatrix}10 & 5 & 40 & 20\end{bmatrix}$.
- **Diamond expressie:** $e_6 = \Diamond e_2 = \Diamond P_2$, alle tweede componenten van de featurevectoren van de buren worden opgeteld. We kunnen dit dan ook uitschrijven voor node 0 bijvoorbeeld als $e_6(G)(\chi)(0) = e_2(G)(\chi)(1) + e_2(G)(\chi)(2) = 1 + 8 = 9$ .De output van de expressie e_6 is $\begin{bmatrix}9 & 14 & 7 & 9\end{bmatrix}$.

VII. Probleemstelling

Een Message Passing Neural Network (MPNN) vereist dat operaties in een strikte volgorde worden uitgevoerd. Dit betekent dat de volgorde van bewerkingen vooraf vastligt en niet zomaar veranderd kan worden. Daarentegen biedt MPLang de flexibiliteit om deze operaties in willekeurige volgorde te combineren en toe te passen. Deze eigenschap maakt het gebruik van een MPNN eenvoudiger en de code leesbaarder voor de gebruiker, omdat de gebruiker niet strikt gebonden is aan een vooraf bepaalde volgorde van operaties.

In eerder onderzoek is theoretisch aangetoond dat het mogelijk is om MPLang om te zetten naar een ReLu-MPNN. Het volgende belangrijke aspect is echter de praktische implementatie van deze vertaling. Dit houdt in dat de concepten en methoden die theoretisch zijn beschreven, moeten worden vertaald naar een werkende, praktische toepassing.

VIII. Algoritme

Voor we naar het eigenlijke algoritme kunnen gaan, moeten we eerst enkele andere methods uitleggen. Deze sectie is volledig voortgebracht uit de onderzoekspaper, [2].

A. Concatenatie van twee MPNN-lagen

Concatenatie van lagen, wil eigenlijk zeggen dat we ze samenvoegen. De input blijft dan ook dezelfde als voor één van deze twee lagen, dus de input ariteit blijft hetzelfde. De output wordt dan een combinatie van de output van de ene laag met de andere laag, de output ariteit wordt dan de som van de twee output ariteiten van de lagen die we concateneren.

De concatenatie van twee MPNN-lagen $L = (W_{1L}, W_{2L}, b_L, \sigma)$ en $K = (W_{1K}, W_{2K}, b_K, \sigma)$ gaat als volgt, de W_1 en W_2 van beide lagen worden verticaal gestapeld. Dan krijgen we voor $K|L$ de W_1 als $\begin{bmatrix}W_{1L}\\W_{1K}\end{bmatrix}$ en voor W_2 dan $\begin{bmatrix}W_{2L}\\W_{2K}\end{bmatrix}$. De bias wordt horizontaal naast elkaar geplaatst $\begin{bmatrix}b_L\\b_K\end{bmatrix}$.

Neem MPNN-lagen $L = (\begin{bmatrix}1 & 2\end{bmatrix}, \begin{bmatrix}3 & 4\end{bmatrix}, 0, \sigma)$ en $K = (\begin{bmatrix}5 & 6\end{bmatrix}, \begin{bmatrix}7 & 8\end{bmatrix}, 1, \sigma)$. De concatenatie $L|K$ wordt dan

$W_1 = \begin{bmatrix}1 & 2\\ 5 & 6\end{bmatrix}$, voor $W_2 = \begin{bmatrix}3 & 4\\ 7 & 8\end{bmatrix}$, de bias $b = \begin{bmatrix}0\\ 1\end{bmatrix}$ en als activationfunctie krijgen we σ

B. Parallelle compositie van twee MPNN-lagen

We willen parallel twee lagen gaan uitvoeren. De input wordt ook verticaal gestapeld en doorloopt zo de laag. Dus de input ariteiten worden opgeteld en de output ariteiten ook.

Bij de parallelle compositie van twee MPNN-lagen L en K zijn W_1 en W_2 gelijk aan de diagonaal opeengestapelde vectoren van de lagen. Dan krijgen we bij $L||K$ voor $W_1 = \left[\begin{array}{c|c} W_{1L} & 0 \\ \hline 0 & W_{1K}\end{array}\right]$, equivalent hieraan kunnen we ook W_2 vormen. De bias b wordt horizontaal gestapeld, dus we krijgen $\begin{bmatrix}b_L\\ b_K\end{bmatrix}$.

Neem de MPNN-lagen L en K uit de sectie hierboven. De parallelle compositie is dan als volgt, $(L||K) = (\begin{bmatrix}1 & 2 & 0 & 0\\ 0 & 0 & 5 & 6\end{bmatrix}, \begin{bmatrix}3 & 4 & 0 & 0\\ 0 & 0 & 7 & 8\end{bmatrix}, \begin{bmatrix}0\\ 1\end{bmatrix}, \sigma)$

C. Omzetting naar ReLu-MPNN

We willen steeds een ReLu-MPNN bekomen, daarom moeten we soms lagen gaan omvormen naar ReLu-lagen. Als er een id-laag L is en een σ-laag K in een sequentie $L;K$, dan bestaat er een sequentie $L';K'$ waarbij L' een ReLu-laag is en K een σ-laag.

Dan wordt L' de concatentatie van L en $-L$ met als activation functie ReLu, dit is gebaseerd op de identiteitsfunctie van ReLu namelijk $x = ReLu(x) - ReLu(-x)$. De laag K' is de laag K, waarbij W_1 en W_2 worden $[W_1|-W_1]$ en $[W_2|-W_2]$.

Stel lagen $L = (\begin{bmatrix}1 & 2\end{bmatrix}, \begin{bmatrix}3 & 4\end{bmatrix}, 0, id)$ en $K = (\begin{bmatrix}5\end{bmatrix}, \begin{bmatrix}7\end{bmatrix}, 1, \sigma)$, de sequentiële compositie $L;K$ is een MPNN. We vormen nu deze MPNN om naar een ReLu-MPNN met $L';K'$. Voor L' krijgen we $L' = (L| - L) = (\begin{bmatrix}1 & 2\\ -1 & -2\end{bmatrix}, \begin{bmatrix}3 & 4\\ -3 & -4\end{bmatrix}, \begin{bmatrix}0\\ 0\end{bmatrix}, ReLu)$ en voor $K' = (\begin{bmatrix}5 & -5\end{bmatrix}, \begin{bmatrix}7 & -7\end{bmatrix}, 1, \sigma)$

D. Concatenatie van twee ReLu-MPNN's

Als we twee ReLu-MPNN's E en D willen concateneren, moeten we eerst zorgen dat ze beide evenveel aantal lagen hebben. Stel n een natuurlijk getal, dan moeten beide n lagen hebben.

Één of meerdere lagen toevoegen kan door voor de σ-laag één of meerdere ReLu-lagen toe te voegen, van de vorm $L = (W_{1id}, \vec{0}, \vec{0}, ReLu)$, met W_{1id} de eenheidsmatrix. Dit steunt op de idempotentie van ReLu, na de eerste uitvoering van ReLu zullen de volgende geen veranderingen meer brengen in de data.

Als de MPNN waar we lagen willen aan toe voegen uit één id-laag bestaat, dan wordt het anders gedaan. Dan voegen we eerst achteraan een id-laag toe van de vorm $L = (1, 0, 0, id)$. Daarna creëren we volgens de vorige sectie VIII-C een Relu-MPNN. Vervolgens kunnen we lagen toevoegen zoals hierboven.

Als n een natuurlijk getal het aantal lagen voorstelt, dan bij $n \geq 2$ is de concatenatie van twee n-lagen ReLu-MPNN's $E = E_1; \ldots; E_n$ en $D = D_1; \ldots; D_n$ gelijk aan $(E_1|D_1); (E_2||D_2); \ldots; (E_n||D_n)$. De eerste laag van de MPNN's kunnen dus volgens de concatenatie van twee lagen worden uitgevoerd. De volgende lagen zullen met parallelle compositie worden samengevoegd.

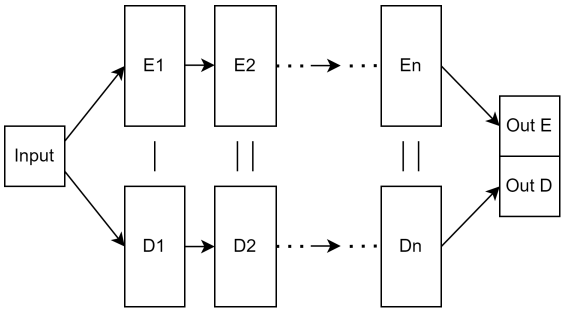


Fig. 2. Concatenatie van twee ReLu-MPNN's

In bovenstaande figuur zien we hoe een concatenatie van twee ReLu-MPNN's werkt. We hebben $E = E_1; \ldots; E_n$ en $D = D_1; \ldots; D_n$, waarbij de input en outputs door de sequentie compositie overeenkomen. De eerste laag krijgt de input zoals voorheen, door de concatenatie is de output een concatenatie van de output van E_1 en D_1. Deze concatenatie is de input van de tweede laag, daarom is hier een parallelle compositie. De output van E_1 gaat vervolgens naar E_2 en zo verder, equivalent hieraan gebeurt dit ook bij D_1 en D_2.

E. MLPang naar ReLu-MPNN

Elke ReLu-MPLang expressie e kan worden vertaald naar een ReLu-MPNN E, een sequentie van MPNN-lagen. Deze MPNN bestaat, indien er één of meerdere subexpressies zijn, uit de ReLu-MPNN van de subexpressie en nog een laag die wordt toegevoegd. Na het toevoegen wordt de MPNN vertaald naar een ReLu-MPNN.

We zien bij de σ-functie telkens de id-functie terugkeren, dit is $f(x) = x$. Voor de vectoren zijn er in sommige gevallen nulvectoren $\vec{0}$, deze zijn matrices van $d \times r$ voor W_1en W_2en voor de bias $r \times 1$.

Er zijn veel lagen die voor W_1, W_2 en de bias een getal hebben en geen grotere vector. Als een expressie een subexpressie heeft, zal deze MPNN van de subexpressie een output geven van output ariteit 1. Omdat MPLang expressies een GFMT definiëren van type $d \to 1$. De volgende laag heeft als input ariteit dan ook 1, dus de matrices zijn 1×1.

- **Constante:** als $e = a$, dan is de output $E = (\vec{0}, \vec{0}, a, id)$ We zien dat de feature vectoren van de huidige node en

de buren vermenigvuldigd worden met nul, dus enkel de bias, de constante, blijft over.

- **Component:** als $e = P_i$, dan is de output $E = (W_1, \vec{0}, \vec{0}, id)$. De W_1 is de identiteitsmatrix voor het i-de component, dus overal 0 met op de i-de plaats 1. De som van de buren wordt vermenigvuldigd met nul en de bias is ook nul.
- **Scalair product:** als $e = ae_1$, dan is de output $E = E_1; \ldots; E_n; (a, 0, 0, id)$. De features van de huidige node die voortkomen uit de subexpressie e_1 worden met a vermenigvuldigd. De som van de buren wordt vermenigvuldigd met nul en de bias is ook nul.
- **Som:** als $e = e_1 + e_2$, dan is de output $E = (E_1|E_2); ((1,1),(0,0),0,id)$. De resulterende MPNN's uit de subexpressies e_1 en e_2 zijn E_1 en E_2. De concatenatie van deze MPNN's vormen de MPNN waaraan de laag van de som wordt toegevoegd. Deze laag zorgt ervoor dat de resultaten van de subexpressies e_1 en e_2 voor de huidige node worden opgeteld.
- **Functie:** als $e = f(e_1)$, dan is de output $E = E_1; \ldots; E_n; (1, 0, 0, f)$. We behouden het resultaat van de huidige node, de som van de buren en de bias zijn nul. Op dit resultaat wordt de functie f uitgevoerd.
- **Diamond:** als $e = \Diamond e_1$, dan is de output $E = E_1; \ldots; E_n; (0, 1, 0, id)$. De resultaten voor de huidige node van de subexpressie en de bias worden 0. De som van resultaten van de buren wordt het resultaat van deze MPNN.

F. Voorbeeld vertaling

We hebben alle nodige informatie om een vertaling van MPLang naar MPNN te maken. We doorlopen even het algoritme aan de hand van een voorbeeld, stel we willen $e_1 = 5 + (5 + P_2)$ omvormen naar een ReLu-MPNN voor de input zoals in figuur 1. We krijgen een expressieboom als volgt:

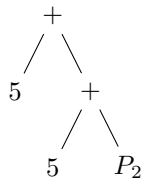


Fig. 3. Expressieboom MPLang expressie $e_1 = 5 + (5 + P_2)$

We zullen deze boom postorder doorlopen. Bij de laatste ontmoeting, worden de MPNN's gecreëerd. We zullen dus van blad 5 in het rechterdeelboom, naar blad 5 in de rechterdeelboom gaan, naar blad P_2, naar de som van deze bladeren, om als laatste de hele som van beide deelbomen te maken. Dus eerst worden de bladeren omgezet, we krijgen links $E_1 = (\begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, 5, id)$.

Voor de rechterboomdeel moeten we eerst 5 en P_2 omvormen en de som van nemen, we krijgen voor 5 terug E_1 en voor P_2 krijgen we $E_2 = (\begin{bmatrix}0 & 1 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, 0, id)$. Nu we de bladeren hebben kunnen we recursief naar boven opwerken, eerst de som van de twee bladeren in de rechter deelboom.

Bij de som MPLang expressie krijgen we $E_3 = (E_1|E_2); (\begin{bmatrix}1 & 1\end{bmatrix}, \begin{bmatrix}0 & 0\end{bmatrix}, 0, id)$, dus we doen eerst de concatenatie van E_1 en E_2. Aangezien ze beide uit één laag bestaan, concateneren we gewoon de lagen volgens sectie VIII-A en dat geeft:

$$(E_1|E_2) = (\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}5\\ 0\end{bmatrix}, id$$

Nu we de MPNN van de concatenatie hebben moeten we E_3 omvormen naar een ReLu-MPNN volgens sectie VIII-C voor we verder gaan. We krijgen:

$$E_3 = (\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & -1 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}5\\ 0\\ -5\\ 0\end{bmatrix}, ReLu);$$
$$(\begin{bmatrix}1 & 1 & -1 & -1\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, 0, id)$$

Nu kunnen we als laatste de som gaan uitvoeren op de twee resulterende MPNN's van de deelbomen. We krijgen de MPNN $E_4 = (E_1|E_3); (\begin{bmatrix}1 & 1\end{bmatrix}, \begin{bmatrix}0 & 0\end{bmatrix}, 0, id)$ voor de som. Eerst gaan we de concatenatie van de twee ReLu-MPNN's E_1 en E_3 uitvoeren.

We moeten eerst evenveel lagen creëren, dus bij E_1 voegen we een laag toe zoals in sectie VIII-D. We voegen eerst een id-laag toe en krijgen $E_1 = (\begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\end{bmatrix}, 5, id); (1, 0, 0, id)$. Deze id-laag heeft als input dimensie 1, want we krijgen slechts één feature per node terug. Nu wijzigen we deze MPNN naar een ReLu-MPNN, dus dan

$$E_1 = (\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}5\\ -5\end{bmatrix}, ReLu);$$
$$(\begin{bmatrix}1 & -1\end{bmatrix}, \begin{bmatrix}0 & 0\end{bmatrix}, 0, id)$$

.

Nu kunnen we ReLu-MPNN E_1 en E_3 gaan concateneren, volgens sectie VIII-D. De eerste laag wordt een concatenatie van de eerste lagen volgens sectie VIII-A, de tweede een parallelle compositie van de tweede lagen volgens sectie VIII-B. We krijgen $(E_1|E_3) =$

$$(\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & -1 & 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}5\\ -5\\ 5\\ 0\\ -5\\ 0\end{bmatrix}, ReLu);$$

$$(\begin{bmatrix}1 & -1 & 0 & 0 & 0 & 0\\ 0 & 0 & 1 & 1 & -1 & -1\end{bmatrix}, \begin{bmatrix}0 & 0 & 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & 0 & 0\end{bmatrix}, \begin{bmatrix}0\\ 0\end{bmatrix}, id)$$

Nu kunnen we de laag van de som expressie toevoegen en omzetten naar een ReLu-MPNN zoals in sectie VIII-C, dan krijgen we als resultaat: $E_4 =$

$$(\begin{bmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&1&0&0\\0&0&0&0\\0&-1&0&0\end{bmatrix},\begin{bmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\end{bmatrix},\begin{bmatrix}5\\-5\\5\\0\\-5\\0\end{bmatrix},ReLu);$$

$$(\begin{bmatrix}1&-1&0&0&0&0\\0&0&1&1&-1&-1\\-1&1&0&0&0&0\\0&0&-1&-1&1&1\end{bmatrix},\begin{bmatrix}0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\end{bmatrix},$$

$$\begin{bmatrix}0\\0\\0\\0\end{bmatrix},ReLu);(\begin{bmatrix}1&1&-1&-1\end{bmatrix},\begin{bmatrix}0&0&0&0\end{bmatrix},0,id)$$

Om te testen kunnen we eerst met MPLang de berekening maken. We krijgen voor $e_1 = 5 + (5 + P_2))$, eerst alle tweede elementen $\begin{bmatrix}2&1&8&4\end{bmatrix}$. Daarna de som van deze elementen met 5, dan krijgen we $\begin{bmatrix}7&6&13&9\end{bmatrix}$. Als laatste voeren we nog de som uit, alles plus vijf en bekomen als output $\begin{bmatrix}12&11&18&14\end{bmatrix}$.

Nu met onze ReLu-MPNN, kunnen we laag per laag gaan berekenen. Voor de eerste laag op node 0, krijgen we:

$$ReLu(\begin{bmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&1&0&0\\0&0&0&0\\0&-1&0&0\end{bmatrix}\cdot\begin{bmatrix}12\\2\\7\\3\end{bmatrix}+\begin{bmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}10\\1\\15\\2\end{bmatrix}$$

$$+\begin{bmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}3\\8\\1\\1\end{bmatrix}+\begin{bmatrix}5\\-5\\5\\0\\-5\\0\end{bmatrix})=ReLu(\begin{bmatrix}5\\-5\\5\\2\\-5\\-2\end{bmatrix})=\begin{bmatrix}5\\0\\5\\2\\0\\0\end{bmatrix}$$

Als we dit voor alle nodes doen, dan na laag 1 krijgen we de feature map $\begin{bmatrix}\begin{bmatrix}5\\0\\5\\2\\0\\0\end{bmatrix}&\begin{bmatrix}5\\0\\5\\1\\0\\0\end{bmatrix}&\begin{bmatrix}5\\0\\5\\8\\0\\0\end{bmatrix}&\begin{bmatrix}5\\0\\5\\4\\0\\0\end{bmatrix}\end{bmatrix}$

Deze feature map wordt de input van de tweede laag, we berekenen deze ook op node 0:

$$ReLu(\begin{bmatrix}1&-1&0&0&0&0\\0&0&1&1&-1&-1\\-1&1&0&0&0&0\\0&0&-1&-1&1&1\end{bmatrix}\cdot\begin{bmatrix}5\\0\\5\\2\\0\\0\end{bmatrix}$$

$$+\begin{bmatrix}0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}5\\0\\5\\1\\0\\0\end{bmatrix}+\begin{bmatrix}0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\\0&0&0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}5\\0\\5\\8\\0\\0\end{bmatrix}$$

$$+\begin{bmatrix}0\\0\\0\\0\end{bmatrix})=ReLu(\begin{bmatrix}5\\7\\-5\\-7\end{bmatrix})=\begin{bmatrix}5\\7\\0\\0\end{bmatrix}$$

Als we dit voor alle nodes doen, dan na laag 2 krijgen we de feature map $\begin{bmatrix}\begin{bmatrix}5\\7\\0\\0\end{bmatrix}&\begin{bmatrix}5\\6\\0\\0\end{bmatrix}&\begin{bmatrix}5\\13\\0\\0\end{bmatrix}&\begin{bmatrix}5\\9\\0\\0\end{bmatrix}\end{bmatrix}$

Deze feature map wordt de input van de derde en laatste laag, we berekenen deze ook op node 0:

$$id(\begin{bmatrix}1&1&-1&-1\end{bmatrix}\cdot\begin{bmatrix}5\\7\\0\\0\end{bmatrix}+\begin{bmatrix}0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}5\\6\\0\\0\end{bmatrix}$$

$$+\begin{bmatrix}0&0&0&0\end{bmatrix}\cdot\begin{bmatrix}5\\13\\0\\0\end{bmatrix}+0)=12$$

De hele feature map uit de ReLu-MPNN van de MPLang expressie $e_1 = 5 + (5 + P_2)$, is dan: $\begin{bmatrix}12&11&18&14\end{bmatrix}$. Dit komt overeen met de berekening van de MPLang expressie, dus de omzetting klopt.

IX. Implementatie keuze

De keuze voor Python als programmeertaal voor de implementatie is gebaseerd op verschillende factoren, waaronder de populariteit van Python in de wetenschap en machine learning. Ook de ondersteuning voor numerieke berekeningen in NumPy en de beschikbaarheid van populaire libraries voor neurale netwerken zoals PyTorch Geometric en Tensorflow. [3], [4]

NumPy is een package voor numerieke berekeningen in Python en biedt een ndarray object dat meerdere-dimensionale arrays kan manipuleren. Deze ndarray's zijn geschikt voor het werken met vectoren en matrices, die nodig zijn voor de

MPNN's op te stellen en te testen. [3]

PyTorch Geometric en Tensorflow zijn open source machine learning libraries die steeds populairder worden door hun flexibiliteit en efficiëntie bij het bouwen en trainen van neurale netwerken. PyTorch Geometric is een uitbreiding van PyTorch en specifiek voor het werken met GNN's. Het voorziet tools voor het opzetten, trainen en evalueren van GNN's. [4]

Door de implementatie in Python met behulp van NumPy, wordt een basis gelegd voor verdere integratie van MPLang. De structuur van de output MPNN's is gebaseerd op bestaande PyTorch Geometric modellen. Zo is het eenvoudig te exporteren en te gebruiken in de library. Een verdere uitbereiding kan zijn van de implementatie rechtstreeks te gaan implementeren in Pytorch Geometric, zodanig dat derden MPLang ook kunnen gaan gebruiken.

X. Implementatie

De implementatie van de vertaling is opgesteld in een klassenstructuur, als volgt:

- **Activation Function:** een klasse ontworpen om functie objecten aan te maken en uit te voeren. Zo kunnen deze eenvoudig in de MPNN-lagen worden gebruikt.
- **Graph:** een klasse ontworpen om graafobjecten te creëren die essentieel zijn voor het testen en valideren van de implementatie. Een graaf wordt gedefinieerd door nodes en edges en de bijhorende feature maps. Deze klasse laat toe diverse grafen te genereren die als input kunnen dienen voor onze neurale netwerken. De Graph klasse ondersteunt functies voor het ophalen van data over het object. Het object kan aangemaakt worden door een adjacencymatrix en feature map door te geven.
- **MPLang Expression:** is een abstracte klasse die dient als basis voor alle expressieklassen binnen MPLang. Elke expressie in MPLang wordt uitgedrukt in een eigen klasse die specifieke functies bevat, zoals een testfunctie om de expressie te controleren en een functie om een MPNN te genereren. De afgeleide klassen zijn: number, component, sum, diamond, scalair product en function.
- **MPNN:** deze klasse beheert een lijst van alle MPNN-laagobjecten die door MPLang-expressies worden gegenereerd. Deze klasse dient als de output van de MPLang-transformatie en bevat de complete structuur van het neurale netwerk. Een MPNN-laagobject houdt de vier elementen bij van een laag, $w_1, W_2, bias, \sigma$, deze kunnen ook uitgevoerd worden. Een MPNN heeft ook een MPNN-operator object en deze kan de mpnn omzetten naar een ReLu-MPNN en een concatenatie van twee MPNN's uitvoeren.

Belangrijk is dat de bias aangemaakt wordt in de dimensies $1 \times r$ en in de laag wordt deze getransponeerd en opgeslagen met de dimensie $r \times 1$. Zodat deze juist wordt opgeslagen voor de berekeningen en in de juiste vorm voor de output MPNN. De featurevectoren geven we ook mee als rijvectoren en slaan deze wel zo op, bij de berekening wordt deze getransponeerd. Dit is omdat het eenvoudiger aan te maken is op deze manier en met een eenvoudige transponeer functie van NumPy op te lossen was.

A. Algoritme in de implementatie

Belangrijk is dat de code in deze paper een verkorte versie is, er is een soort pseudocode. Aangezien het te omvangrijk was om volledig te implementeren. Er worden enkele afkortingen gebruikt voor namen, het definiëren van types voor argumenten wordt weggelaten en self wordt waar kan ook weggelaten. De logica is wel volledig behouden.

Eerst moeten we een expressie vormen. Dit kan door objecten aan te maken van de afgeleide MPLang Expressies. De volgende expressies:

$e_1 = 5$
$e_2 = P_2$
$e_3 = 5 \cdot e_2 = 5 \cdot P_2$
$e_4 = e_1 + e_2 = 5 + P_2$
$e_5 = f(e_2) = f(P_2)$, met f(x) = x
$e_6 = \Diamond e_4 = \Diamond(5 + P_2)$
$e_7 = e_1 + e_4 = 5 + (5 + P_2)$
$e_8 = \Diamond(2 \cdot (e_7)) = \Diamond(2 \cdot (5 + (5 + P_2)))$

Deze MPLang expressies kunnen we in de implementatie vormen als:

```
exp1 = number(5)
exp2 = component(2)
exp3 = scalairProduct(exp2, 5)
exp4 = sum(exp1, exp2)
exp5 = function(activationFunction("id"), exp2)
exp6 = diamond(exp4)
exp7 = sum(exp1, exp4)
exp8 = diamond(scalairProduct(exp7, 2))
```

Listing 1. Code of MPLang expressions

Op deze expressies kunnen we dan de functie gaan oproepen 'createMPNN()', die heel het algoritme in gang zal zetten.

Voor het algoritme in de implementatie te verduidelijken, vallen we terug op het voorbeeld van de vertaling in sectie VIII-F Voorbeeld vertaling. Dat is in bovenstaande expressies, expressie 7.

De functie wordt op een som expressie opgeroepen, hierbij ziet deze functie er als volgt uit:

```
def createMPNN(d) -> MPNN:
    mpnn1 = expression1.createMPNN(d)
    mpnn2 = expression2.createMPNN(d)
    self.addMPNN(mpnn1.concatenation(mpnn2))

    self.getMPNN().
        addLayer(MPNNLayer(np.array([1,1]),
                 np.array([0,0]),
                 np.array([0]),
```

```
                activFunction('id')))
    return self._mpnn
```

Listing 2. Code of createMPNN() in Sum MPLang expression

We geven het aantal features, de dimensie, mee aan de expressie. Zo kunnen de matrices correct gevormd worden, met de input ariteit. Dit alles gebeurt volgens sectie VIII-E.We zien dat eerst op de subexpressies dezelfde functie wordt opgeroepen, deze output MPNN's zijn nodig voor de MPNN te vormen.

De eerste subexpressie $e_1 = 5$, geeft een eenvoudige MPNN met één laag terug. De andere subsexpressie $e_4 = 5 + P_2$ geeft de som van zijn eigen subexpressies $e_1 = 5$ en $e_2 = P_2$ terug. Aangezien het een som is, is dit is met dezelfde code als hierboven.

We kijken naar de subexpressie e_4, deze krijgt van zijn subexpressies twee MPNN's terug van telkens 1 laag. Dit is omdat de number en component MPLang expressies zijn de bladeren, en hebben dus maar één laag. We voegen aan onze som MPLang expressie de concatenatie van onze twee MPNN's toe. De uitvoering van de concatenatie van twee MPNN's gaat als volgt:

```
def concatenation(mpnn1, mppn2) -> MPNN:
    new_mpnn = MPNN([])
    diff =
        len(mpnn1.getLayers())
        -len(mpnn2.getLayers())

    if(diff > 0):
        createLayers(mpnn2, diff)
    elif(diff < 0):
        createLayers(mpnn1, diff*(-1))

    for i in range(len(mpnn1.getLayers())):
        if(i == 0):
            new_mpnn.addLayer(
                concatLayers(
                mpnn1.getLayer(i),
                mpnn2.getLayer(i)))
        else:
            new_mpnn.addLayer(
                parComp(
                mpnn1.getLayer(i),
                mpnn2.getLayer(i)))

    return new_mpnn
```

Listing 3. Code van de concatenatie van 2 MPNN's

De implementatie is zoals het algoritme, in sectie VIII-D.Eerst maken we een lege MPNN aan en berekenen het verschil in lagen. Dan gaan we kijken welke van de twee lagen te weinig heeft en deze bij aanmaken indien nodig met 'createLayers(mpnn, diff)'. Daarna gaan we over alle lagen, bij de eerste gaan we beide eerste lagen concateneren met 'concatLayers(layer, layer), daarna alle volgende paren gaan we een parallel composition maken met 'parComp()'. Deze nieuwe lagen voegen we telkens toe aan het nieuwe MPNN-object.

De createLayers werkt als volgt, als er al twee of meer lagen zijn gaan we de laatste laag verwijderen en dan aan de hand van de output shape van de voorlaatste laag ReLu lagen maken en toevoegen. Als er nog maar één laag is, dan gaan we eerst achteraan een id-laag toevoegen en daarna terug deze functie oproepen.

```
def createLayers(mpnn, x):
    lastLayer = mpnn.getLayer(-1)
    lenBias = len(lastLayer.getBias())
    shape = (lenBias, lenBias)

    if(len(mpnn.getLayers()) >= 2):
        mpnn.removeLastLayer()
        for i in range(x):
            mpnn.addLayer(
                MPNNLayer(np.full(shape, 1),
                          np.zeros(shape),
                          np.zeros(lenBias),
                          activFunction('relu')))
        mpnn.addLayer(lastLayer)
    else:
        mpnn.addLayer(
            MPNNLayer(np.full(shape,1),
                      np.zeros(shape),
                      np.zeros(lenBias),
                      activFunction('id')))
        if (x > 1):
            createLayers(mpnn, x-1)
```

Listing 4. Code voor extra lagen toe te voegen

De concatenatie van lagen, dat we nodig hebben voor de concatenatie van MPNN's. Deze functie krijgt de twee lagen en geeft één laag terug. Dit gebeurt volgens het algoritme, in sectie VIII-A.

```
def concatLayers(layer1, layer2) -> MPNNLayer:
    return MPNNLayer(
        np.vstack((layer1.getw1(),
                   layer2.getw1())),
        np.vstack((layer1.getw2(),
                   layer2.getw2())),
        np.concatenate((layer1.getBias(),
                        layer2.getBias()),
                        axis=None),
        layer1.getSigma())
```

Listing 5. Code van de concatenatie van 2 lagen

De nieuwe W_1, W_2 en bias worden aan de hand van NumPy gevormd. Voor W_1 en W_2 gebruiken we 'vstack()', deze functie stapelt de matrices verticaal. Voor de bias gebruiken we 'concatenate()', die de matrices achter elkaar plaatst.

De parallel composition werkt op een gelijkaardige manier, volgens sectie VIII-B in het algoritme. Hierbij gebruiken we ook nog de 'hstack()', die de matrices horizontaal achter elkaar plaatst. De dimensies voor de nulvectoren worden gevormd in een andere functie, daarom is hier een verkorte versie.

```
def parComp(layer1, layer2) -> MPNNLayer:
    w11 = layer1.getw1() #dim = axb
    w12 = layer2.getw1() #dim = cxd
    w21 = layer1.getw2() #dim = exf
    w22 = layer2.getw2() #dim = gxh
    return MPNNLayer(
        np.vstack(
            np.hstack(w11, np.zeros(a, d)),
            np.hstack(np.zeros(c, b),w12)),
        np.vstack((
            np.hstack(w21, np.zeros(e, h)),
            np.hstack(np.zeros(g,f),w22)),
        np.hstack((
            layer1.getBias(),
            layer2.getBias())),
        layer1.getSigma())
```

Listing 6. Code van de parallel composition van 2 lagen

Als laatste hebben we ook nog telkens bij het toevoegen van een laag, gaan zorgen dat het een ReLu-MPNN blijft. Als de laatste laag geen ReLu-laag is, dan vormen we de voorlaatste laag om tot een ReLu-laag en vormen die nieuwe laag om en voegen deze ook toe. Dit is zoals het algoritme, in sectie VIII-C.

```
def addLayer(layer):
    if(len(layers) != 0 and
        layers[-1].getSigma()  != 'relu'):
        operator.createReLu(layer, self)
    else:
        layers.append(layer)

def createReLu(layer, mpnn):
    lastLayer = mpnn.getLayer(-1)
    mpnn.removeLastLayer()
    mpnn.addLayer(concatLayers(
        MPNNLayer(lastLayer.getw1(),
                  lastLayer.getw2(),
                  lastLayer.getBias(),
                  activFunction('relu')),
        MPNNLayer(-1*lastLayer.getw1(),
                  -1*lastLayer.getw2(),
                  -1*lastLayer.getBias(),
                  activFunction('relu'))))

    mpnn.addLayer(MPNNLayer(
        np.concatenate((layer.getw1(),
                        -layer.getw1()),
                        axis=None),
        np.concatenate((layer.getw2(),
                        -layer.getw2()),
                        axis=None),
        layer.getBias(),
        layer.getSigma()))
```

Listing 7. Code voor een laag toe te voegen

XI. Bespreking en evaluatie

De implementatie is tot een goed einde gebracht, alle elementen en heel het algoritme zijn geïmplementeerd. Helaas was het niet meer mogelijk een implementatie in een bestaande library te maken van neurale netwerken.

Een verdere mogelijke uitbreiding is dus het implementeren van deze vertaling in een library. Dit zou mogelijk kunnen zijn, aangezien de MPNN gestructureerd is zoals in deze libraries. Een lijst van lagen, die enkel de vier elementen bijhouden, is de output van een MPLang expressie.

Een handige extra tool zou nog een docker container zijn geweest, zodat de implementatie sneller en eenvoudiger te testen was. Nu is het enkel mogelijk door in de test file zelf expressies uit te schrijven.

De code is gestructureerd, maar ik botste op enkele dingen. Vooral in Python was het moeilijker om object gericht te gaan programmeren. Dit kan komen, doordat ik dit niet eerder uitgebreid had gedaan in Python.

Verder ben ik zeer tevreden met het resultaat. Het was een leerzaam proces, om eerst alles onder de knie te krijgen. Het was zeer interessant om aan de hand van een doctoraat een bachelorproef mogen te maken. Dit gaf me de kans om een bijdrage te leveren in een onderzoek. Hierdoor heb ik ook nauw kunnen samenwerken met Juno Steegmans, wat heel leerrijk was.

Conclusie

De implementatie van de vertaling van MPLang expressie naar een MPNN, is in python geschreven. Expressies kunnen gebouwd worden, door het aanmaken van het MPLang expressie objecten. Daarna kunnen deze objecten hun MPNN ontwikkelen aan de hand van het algoritme en geven ze deze terug. MPLang expressies en MPNN zijn zo opgebouwd en gemaakt, dat ze eenvoudig te testen zijn. Als je de expressie en de MPNN evalueert, zullen ze dezelfde output op de input graaf en feature map genereren.

De implementatie van het algoritme is afgewerkt, maar er zijn dus nog enkele uitbreidingen die het meer volmaakt hadden kunnen maken. Ook de optimalisatie van het algoritme in de implementatie had de redundantie kunnen verlagen.

Aan het begin ging het vlot met nieuwe dingen bij te leren, daarna ging het iets moeizamer. De deadlines van de tussentijdse presentatie, zorgden ervoor dat het werk weer vlot verliep. Tijd verdelen en inschatten is een groot werkpunt. Het bijhouden van tijden in de timesheet, was dan ook iets dat ik snel vergat.

Dankbetuiging

Ik wil graag Juno Steegmans bedanken voor haar voortdurende steun en waardevolle adviezen tijdens onze wekelijkse meetings. Ook wil ik mijn oprechte waardering uiten aan Prof. dr. Jan Van de Bussche, mijn promotor, voor zijn begeleiding en expertise.

Niet te vergeten mijn vrienden en familie, die in mij geloofden. Ze zijn een steun en toeverlaat geweest. Dankzij deze hulp en steun is mijn bachelorproef succesvol afgerond.

REFERENCES

[1] William L. Hamilton. Graph Representation Learning. Synthesis Lectures on Artificial Intelligence and Machine Learning, Vol. 14, No. 3 , 2020, pp. 47-73.

[2] F. Geerts, J. Steegmans, J. Van den Bussche, On the expressive power of message-passing neural networks as global feature map transformers, 2022, arXiv:2203.09555.

[3] Harris, C.R., Millman, K.J., van der Walt, S.J. et al. Array programming with NumPy. Nature 585, 2020, pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[4] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035. Curran Associates, Inc. Retrieved from http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[5] Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M. and Monfardini, G., The Graph Neural Network Model, 2009, IEEE Transactions on Neural Networks, Vol 20(1), pp. 61–80.